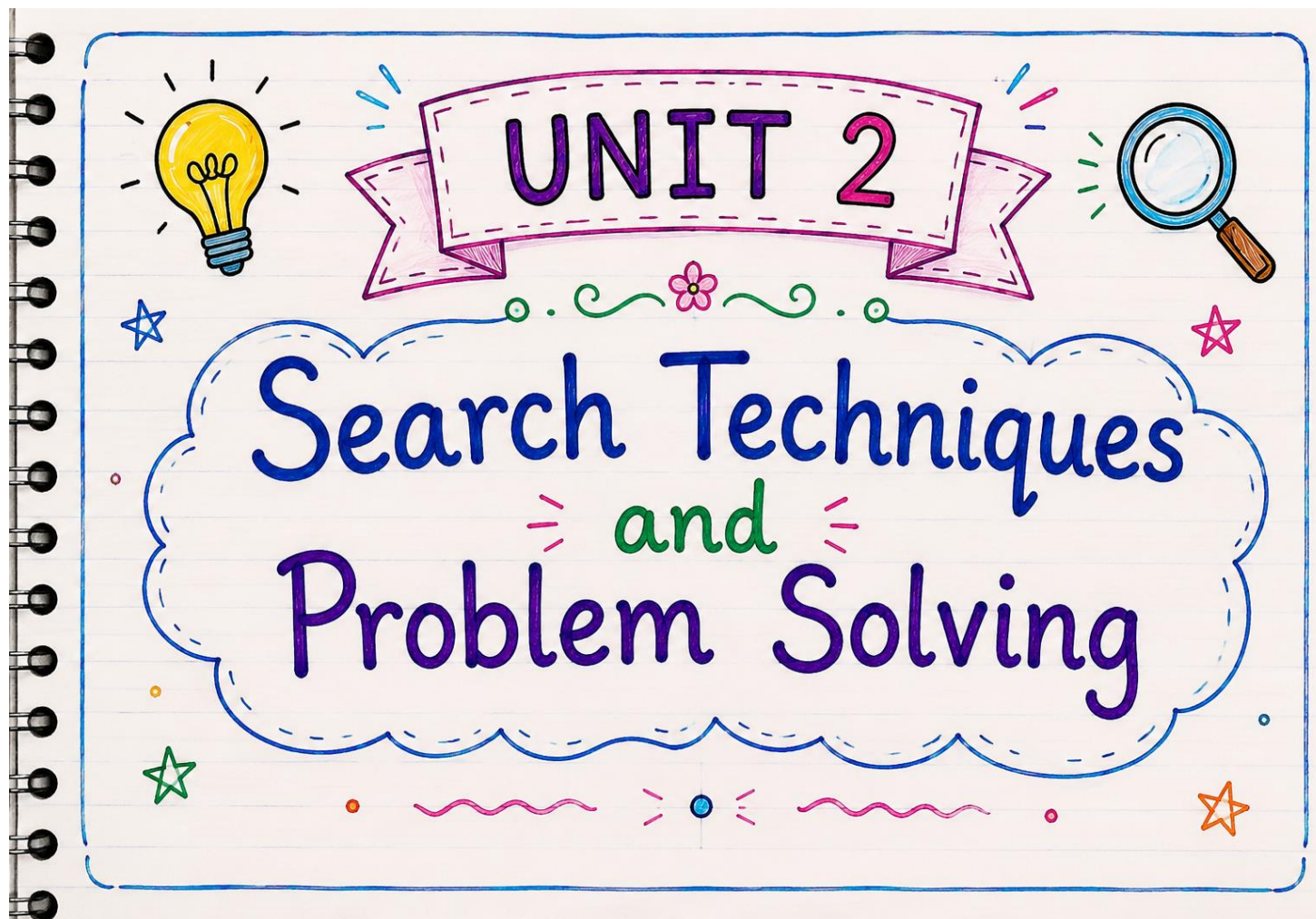




Q.3 (7 Marks) Illustrate Breadth First Search (BFS) with an example. Page-3

Ans: Breadth First Search (BFS) is a graph traversal algorithm. It explores the graph level by level (breadth-wise) starting from the source node. It uses a Queue (FIFO) to keep track of nodes to be visited.

Example Graph:



Given:

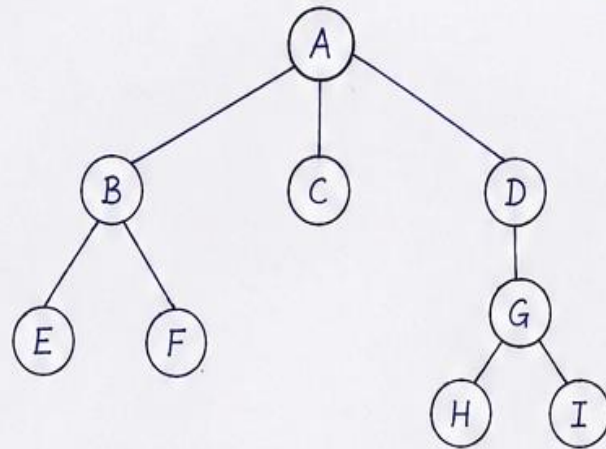
• Vertices : {A B C D E F G}

Q.2 (7 Marks) Demonstrate Depth First Search (DFS) with an example.

Page-2

Ans: Depth First Search (DFS) is an uninformed search algorithm. It explores as deep as possible along one branch before backtracking to explore other branches.

Example: Consider the following graph.



Goal: Find node I

() → Node

— → Edge

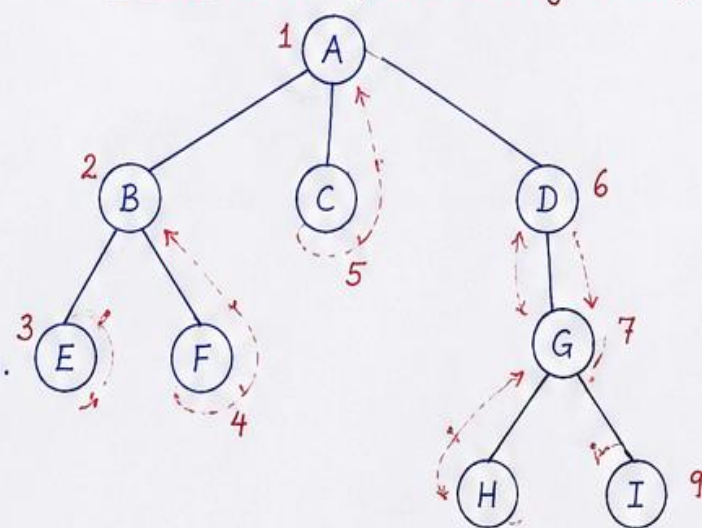
Step-by-Step DFS Traversal (Start from A)

Step	Stack (Top → Right)	Action	Visited Order
1	A	Push A (start)	A
2	D, C, B	Pop A, push its children (B,C,D)	A
3	D, C, F, E	Pop B, push its children (E,F)	A, B
4	D, C, F	Pop E (no child)	A, B, E
5	D, C	Pop F (no child)	A, B, E, F
6	D	Pop C (no child)	A, B, E, F, C
7	G	Pop D, push its child (G)	A, B, E, F, C, D
8	I, H	Pop G, push its children (H,I)	A, B, E, F, C, D, G
9	I	Pop H (no child)	A, B, E, F, C, D, G, H
10	—	Pop I → Goal Found	A, B, E, F, C, D, G, H, I

DFS Traversal Order:

A → B → E → F → C → D → G → H → I

Visual DFS (Backtracking shown)



Key Points:

- DFS uses a Stack (LIFO).
- It goes as deep as possible before backtracking.
- Less memory than BFS.
- Not guaranteed to find shortest path.

Applications of DFS:

- * Solving puzzles (e.g., 8-puzzle, maze).
- * Topological sorting.
- * Detecting cycles in a graph.
- * Finding connected components.

Conclusion:

DFS explores a path completely before backtracking. It is useful when solution may be far from the start or when we want to explore all possibilities.



Q.1 (7 Marks) Explain State Space Search with a suitable example.

Page - 1

Ans : State Space Search is a technique used in AI to solve problems by systematically exploring all possible states to find a path from the initial state to a goal state.

1) State Space :

- A state space is the set of all possible states that can be reached from the initial state by applying a sequence of actions.
- It is usually represented as a graph where nodes are states and edges are actions.

2) Components of State Space Search :

- Initial State : The state at which the problem begins.
- Actions : Set of operations that can be applied.
- Transition Model : Result of applying an action to a state.
- Goal Test : Determines whether a state is a goal state.
- Path Cost : Cost of a path is the sum of costs of actions.

Example : (8-Puzzle Problem)

- We have a 3×3 board with 8 tiles and one blank.
- The task is to reach the goal configuration from the start configuration by sliding tiles into the blank space.

Initial State

1	2	3
4	5	6
7	-	8

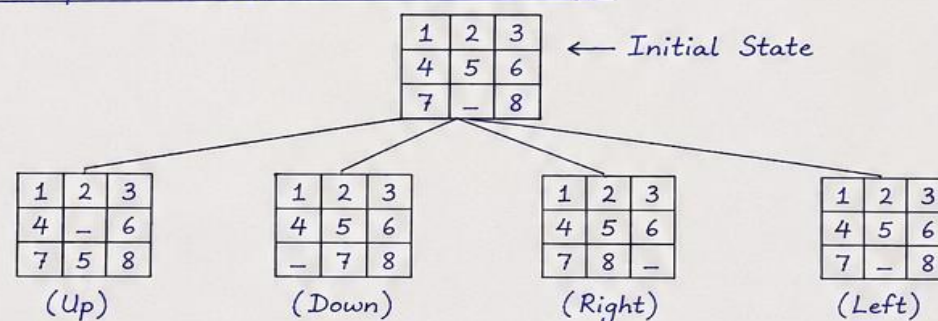
Goal State

1	2	3
4	5	6
7	8	-

Possible Actions

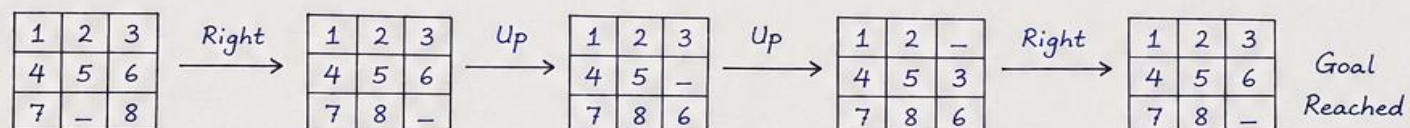
- 1. Up 2. Down
- 3. Left 4. Right

State Space Tree (From Initial State) :



Each node represents a state. From each state, different actions generate new states. The search algorithm explores this tree to find the goal.

Solution Path Example :



Path Cost = 4 (Assuming each move cost = 1)

Conclusion :

State Space Search systematically explores all possible states of a problem using actions. It is the foundation of many search algorithms like BFS, DFS, Uniform Cost Search, etc.

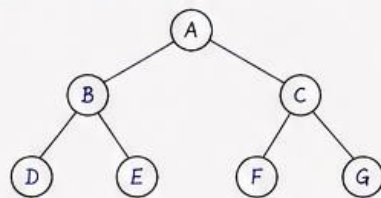
Q.4 (7 Marks) Differentiate Depth First Search (DFS) and Breadth First Search

Page-4

Ans: DFS and BFS are two fundamental uninformed search algorithms used in AI to explore a graph. They differ in the way nodes are visited and in their performance.

Feature	Depth First Search (DFS)	Breadth First Search (BFS)
1. Traversal Style	Explores as deep as possible along one branch before backtracking.	Explores all neighbors level by level (breadth-wise).
2. Data Structure	Stack (LIFO)	Queue (FIFO)
3. Strategy	Go deep first, then backtrack.	Visit all nodes at current level before going to next level.
4. Completeness	Not complete in infinite depth or cycles (may get stuck).	Complete (finds solution if it exists in finite time).
5. Optimality	Not optimal.	Optimal for unweighted graphs (shortest path in terms of edges).
6. Time Complexity	$O(b^m)$	$O(b^d)$
7. Space Complexity	$O(bm)$	$O(b^d)$
8. Best Suited For	Problems with large search space but small solution depth.	Problems where shortest path is required.
9. Memory Usage	Less memory (stores one path from root to current node).	More memory (stores all nodes at current level).
10. Backtracking	Uses backtracking.	No backtracking.
11. Example	Maze solving, Topological sorting, Cycle detection.	Shortest path in unweighted graph, Web crawling, Social network search.

Example Graph (Same for DFS and BFS):



Given:

Start node : A

Goal : (Assume any node)

(We will visit all nodes)

Legend:

○ → Node

— → Edge

Depth First Search (DFS) from A

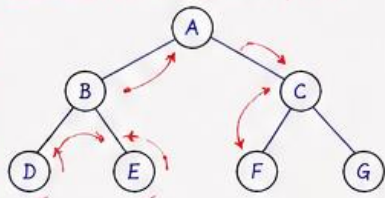
(Using Stack)

Traversal Order : A → B → D → E → C → F → G

Step-by-Step (Stack):

Step	Stack (Top → Right)	Visited	Action
1	A	A	Push A
2	C, B	A	Push B, C
3	C, E, D	A, B	Push D, E
4	C, E	A, B, D	D has no child → Pop D
5	C	A, B, D, E	E has no child → Pop E
6	F, G	A, B, D, E, C	Push F, G
7	F	A, B, D, E, C, G	G has no child → Pop G
8	- (Done)	A, B, D, E, C, G, F	F has no child → Pop F

Visual DFS (One possible path)



Key Differences in One Line:

DFS dives deep into one branch before exploring others.

BFS explores all neighbors at the current level first.

Breadth First Search (BFS) from A

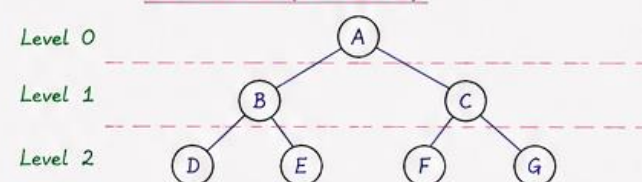
(Using Queue)

Traversal Order : A → B → C → D → E → F → G

Step-by-Step (Queue):

Step	Queue (Front → Rear)	Visited	Action
1	[A]	A	Enqueue A
2	[B, C]	A, B, C	Dequeue A Enqueue B, C
3	[D, E, F, G]	A, B, C, D, E, F, G	Dequeue B Enqueue D, E Dequeue C Enqueue F, G
4	[F, G]	A, B, C, D, E, F, G	
5	[G]	A, B, C, D, E, F, G	No new node
6	[]	A, B, C, D, E, F, G	No new node
7	[]	A, B, C, D, E, F, G	No new node
8	[] (Done)	A, B, C, D, E, F, G	No new node

Visual BFS (Level-wise)



Conclusion:

Both DFS and BFS are important. DFS is useful when memory is limited and solution depth is small, while BFS is optimal for finding the shortest path in an unweighted graph. 😊

1. STATE SPACE SEARCH

Definition:

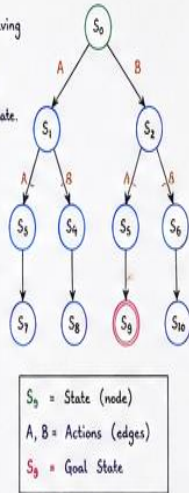
State space search is a method of solving problems by exploring a state space (set of all possible states) to find a path from the initial state to a goal state.

Key Elements:

- States: Possible situations in the problem.
- Initial State: Starting point.
- Actions/Operators: Move from one state to another.
- Goal Test: Checks if a state is goal.
- Path Cost: Cost of a path from start to a state.

Example:

Route finding in maps.



2. PRODUCTION SYSTEMS

Definition:

A production system is a problem-solving model consisting of a set of productions (rules) used to transform one state into another.

Components:

- Working Memory: Set of known facts.
- Production Rules: IF condition THEN action.
- Control Strategy: Decides which rule to apply.

Working:

- Match rules with working memory.
- Select a rule.
- Execute the action (update memory).
- Repeat until goal is reached.



3. PRODUCTION CHARACTERISTICS

Characteristics:

- Modularity: Easy to add new rules.
- Flexibility: Can handle change in knowledge.
- Interactivity: User can add new facts/rules.
- Incrementality: Solution can be improved step by step.
- Transparency: Easy to trace reasoning.
- Non-determinism: More than one rule may apply.

Example:

IF it is raining THEN carry umbrella.
IF umbrella is carried THEN you will not get wet.

4. GENERATE-AND-TEST

Idea:

Generate all possible candidates and test each one to see if it is a goal.

Algorithm:

- Generate a candidate solution.
- Test it.
- If it is goal, return success.
- Else generate next candidate.
- Repeat.

Example:

4-Queens Problem

Generate all placements and test

which ones do not attack each other.



5. HILL CLIMBING

Idea:

Start with an initial state and move to neighboring states that are better. Stop when no better neighbor exists.

Algorithm:

- Choose an initial state.
- Evaluate all neighbors.
- Move to the best neighbor (that is better).
- Repeat until no improvement.

Drawback:

May get stuck in local maxima.



6. BEST-FIRST SEARCH

Idea:

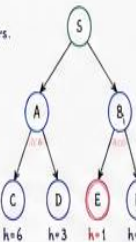
Expand the most promising node according to an evaluation function $h(n)$ (heuristic).

Algorithm:

- Put start node in OPEN list (priority queue).
- Remove node with best $h(n)$.
- Expand it and add its successors.
- Repeat until goal is found.

Example:

Using $h(n)$ = estimated distance to goal.
OPEN list ordered by smallest $h(n)$.



7. PROBLEM REDUCTION

Idea:

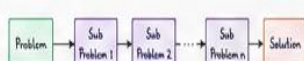
Reduce a complex problem into simpler subproblems and solve them recursively.

Steps:

- Reduce: Transform problem into smaller subproblems.
- Conquer: Solve subproblems.
- Combine: Combine solutions of subproblems.

Example:

Solving a big puzzle by solving smaller sub-puzzles.



8. CONSTRAINT SATISFACTION

Definition:

A CSP is defined by:

- A set of variables.
- Domains for each variable.
- Constraints specifying allowable combinations.

Goal:

Assign values to all variables so that all constraints are satisfied.

Example:

Map Coloring
Color map regions so that no adjacent regions have the same color.



9. MEANS-ENDS ANALYSIS

Idea:

Compare the current state with the goal state and reduce the difference by choosing operators that achieve the differences (subgoals).

Steps:

- Identify the difference between current state and goal.
- Choose an operator that reduces the difference.
- Apply the operator.
- Repeat until goal is reached.



10. BREADTH FIRST SEARCH (BFS)

Idea:

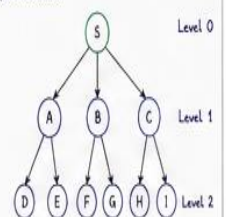
Expand nodes level by level (shallowest first). Uses a FIFO queue.

Algorithm:

- Put start node in queue.
- Remove front node.
- If goal, return success.
- Add all successors to rear of queue.
- Repeat.

Properties:

- Complete: Yes (if branching factor is finite)
- Optimal: Yes (for equal step costs)
- Time: $O(b^d)$
- Space: $O(b^d)$



11. DEPTH FIRST SEARCH (DFS)

Idea:

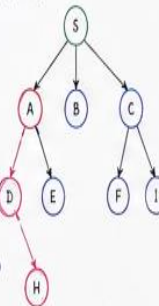
Explore as far as possible along each branch before backtracking. Uses a stack (LIFO).

Algorithm:

- Put start node on stack.
- Pop node from stack.
- If goal, return success.
- Push successors onto stack.
- Repeat.

Properties:

- Complete: No (in infinite depth)
- Optimal: No
- Time: $O(b^m)$
- Space: $O(bm)$



12. HEURISTIC SEARCH TECHNIQUES

Definition:

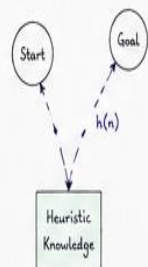
Use heuristic knowledge to guide the search towards goal efficiently.

Common Heuristics:

- $h(n)$: Estimated cost from node n to goal.
- $h(n)$ should be admissible (not overestimate) for optimality.

Types:

- Greedy Best-First Search: Uses only $h(n)$.
- A* Search: Uses $f(n) = g(n) + h(n)$.
- IDA*: Iterative Deepening A*.



13. A* ALGORITHM

Idea:

Find path with minimum total estimated cost $f(n) = g(n) + h(n)$

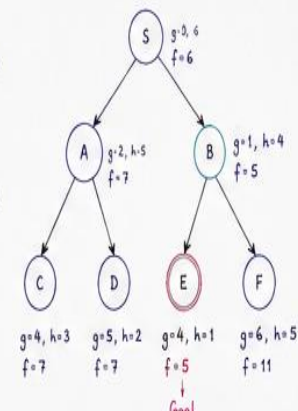
where,

$g(n)$ = cost from start to n

$h(n)$ = estimated cost from n to goal

Algorithm:

- Put start node in OPEN list with $f(n)$.
- Remove node with smallest $f(n)$.
- If goal, return path.
- Expand node and add successors with their $f(n)$.
- Repeat.



PRACTICAL ASPECTS (LAB SESSIONS)

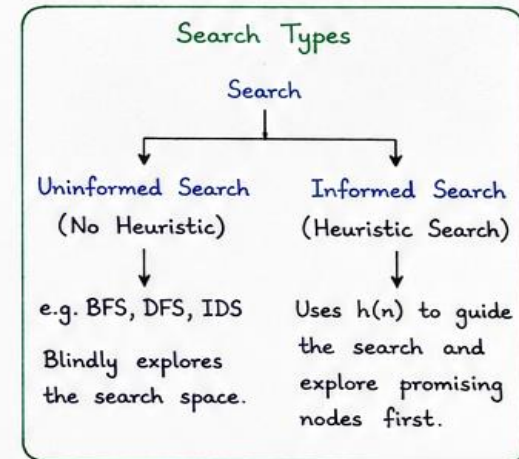
- Implement all above search techniques
- Compare performance (time, memory, optimality)
- Apply to problems like:
 - 8-Puzzle
 - Missionaries and Cannibals
 - N-Queens
 - Map Coloring
- Visualize state space and search trees
- Analyze effects of heuristics



Heuristic Function in Detail

1. What is a Heuristic Function?

- A heuristic function ($h(n)$) is a function used in search algorithms (especially in AI) to **estimate the cost** of reaching the **goal** from a given state n .
- It provides **guidance** to the search process by making it more efficient.
- Heuristics are like "**rules of thumb**" – they don't guarantee the best solution but often lead to **good/optimal** solutions faster.



2. Role of Heuristic Function

- In **uninformed search** (like BFS, DFS), the algorithm has no knowledge of the problem domain; it blindly explores.
- In **informed search** (heuristic search), a heuristic function helps decide which node to explore first, reducing time and memory usage.

3. Example: 8-Puzzle Problem

Goal: Arrange tiles in correct order. Goal State

Initial State				Goal State		
1	2	3	----->	1	2	3
4		6		4	5	6
7	5	8		7	8	

For the given initial state:

$h_1(n) = 2$ (tiles 5 and 6 are misplaced)

$h_2(n) = 1$ (tile 6) + 1 (tile 5) = 2

- Heuristic Functions:
- $h_1(n)$: Number of **misplaced tiles** (tiles not in the correct place).
 - $h_2(n)$: **Manhattan distance** (Sum of distances of each tile from its goal position).

1	2	3	0	0	→ Manhattan distance = 2
4		6	0	1	
7	5	8	1	0	

Both heuristics guide the search, but **Manhattan distance (h_2)** is more accurate and informative.

4. Properties of a Good Heuristic

- Admissibility** → Never overestimates the true cost. (Guarantees optimal solution in A* search).
- Consistency (Monotonicity)** → Estimate should be non-decreasing along a path.

Consistency Condition:

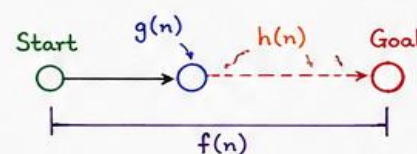
For every node n and its successor n' ,

$$h(n) \leq c(n, n') + h(n')$$

where $c(n, n')$ = cost of step from n to n' .

5. Applications of Heuristic Function

- A* Algorithm** → $f(n) = g(n) + h(n)$
 $g(n)$ = cost so far
 $h(n)$ = estimated cost to goal



- Greedy Best-First Search** → $f(n) = h(n)$
- Pathfinding** → Google Maps, GPS systems.

- Game Playing** → Chess, Tic-tac-toe evaluation functions.
- AI Planning & Optimization Problems.**

★ **Summary:** A heuristic function is a problem-specific knowledge that estimates the cost to reach the goal. It plays a crucial role in informed search algorithms, making them efficient and effective.





HILL CLIMBING

Problem Solving

Hill Climbing is a local search technique that moves from a state to a better neighboring state using a heuristic function.



1. Basic Idea

- Start with an **initial state**.
- Evaluate neighboring states using heuristic $h(n)$.
- Move to the neighbor with **better** (higher or lower) value depending on the problem.
- Repeat until no better neighbor exists.
- The current state is a **local optimum**.

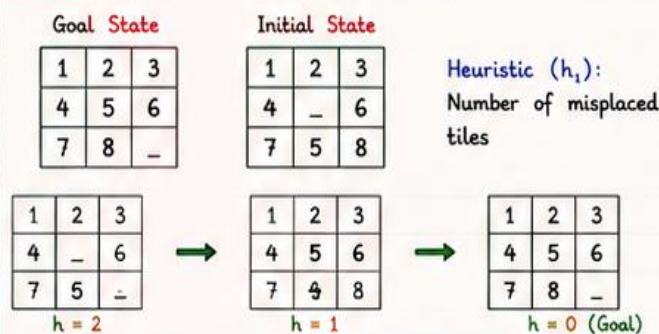
2. How it Works (Steps)

- Choose an initial state n .
- Generate all neighbors of n .
- Evaluate each neighbor using $h(n)$.
- Move to the best neighbor (that improves $h(n)$).
- Repeat steps ②-④ until no neighbor is better.

3. Types of Hill Climbing

- ★ **Simple Hill Climbing**
→ Move to the first better neighbor found.
- ★ **Steepest Ascent Hill Climbing**
→ Move to the neighbor with the best (highest) improvement.
- ★ **Stochastic Hill Climbing**
→ Move to a randomly chosen better neighbor.
- ★ **Sideways Move Hill Climbing**
→ Allow moves to equal neighbors to escape plateaus.
- ★ **Random-Restart Hill Climbing**
→ Run hill climbing multiple times from random initial states.

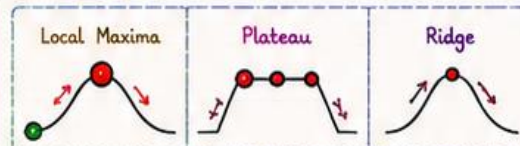
4. Example 1 : 8-Puzzle Problem



At each step we move to the neighbor with lower $h(n)$ until we reach the goal state.

7. Problems / Limitations

- May get stuck at **Local Maxima**.
- May get stuck on **Plateaus** (flat areas).
- May get stuck on **Ridges** (narrow slopes).



★ Conclusion:

Hill Climbing is **simple, fast and memory efficient**. It works well for many problems but does not guarantee **global optimum**. Using techniques like **random restart** or **sideways moves** can improve its performance.



5. Example 2 : Numeric Example

Problem: Maximize the value of $f(x) = -(x-5)^2 + 25$
(This is a hill-shaped function with maximum at $x = 5$)

x	0	1	2	3	4	5	6	7	8	9	10
f(x)	0	9	16	21	24	25	24	21	16	9	0

Start at $x = 0$

Neighbors: $x = -1$ ($f = -4$), $x = 1$ ($f = 9$) → Move to 1

At $x = 1$

Neighbors: 0 ($f = 0$), 2 ($f = 16$) → Move to 2

At $x = 2$

Neighbors: 1 ($f = 9$), 3 ($f = 21$) → Move to 3

At $x = 3$

Neighbors: 2 ($f = 16$), 4 ($f = 24$) → Move to 4

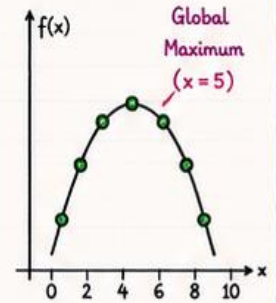
At $x = 4$

Neighbors: 3 ($f = 21$), 5 ($f = 25$) → Move to 5

At $x = 5$

Neighbors: 4 ($f = 24$), 6 ($f = 24$) → No better neighbor

Stop at $x = 5$ (Local Maximum / Global Maximum)



6. Example 3 : N-Queens (4-Queens) using h = number of attacks

Representation: $[a, b, c, d]$ → row position of queen in each column

Goal: Place 4 queens so that no two attack each other ($h = 0$)

Start State: $[1, 3, 0, 2]$ $h = 2$

Neighbors and h values:

Neighbor 1	Neighbor 2	Neighbor 3	Neighbor 4	Neighbor 5
$[2, 3, 0, 2]$	$[0, 3, 0, 2]$	$[1, 2, 0, 2]$	$[1, 3, 1, 2]$	$[1, 3, 2, 2]$
$h = 3$	$h = 1$	$h = 2$	$h = 2$	$h = 2$
Move to best neighbor → $[0, 3, 0, 2]$ ($h = 1$)				
N1	N2	N3	N4	N5
$[1, 3, 0, 2]$	$[0, 2, 0, 2]$	$[0, 1, 0, 2]$	$[0, 3, 1, 2]$	$[0, 3, 2, 2]$
$h = 2$	$h = 2$	$h = 1$	$h = 2$	$h = 2$
Move to best neighbor → $[0, 1, 0, 2]$ ($h = 1$)				
N1	N2	N3	N4	N5
$[1, 1, 0, 2]$	$[0, 0, 0, 2]$	$[0, 2, 0, 2]$	$[0, 1, 1, 2]$	$[0, 1, 2, 2]$
$h = 2$	$h = 2$	$h = 2$	$h = 0$ ✓ Goal	$h = 2$

8. Applications

- Optimization problems
- Game playing
- Scheduling & timetabling
- VLSI design
- Machine learning (parameter tuning)



Finds the best path,
not just any path!

A* Algorithm

"A* = Smart Search
for Shortest Path"

1 What is A*?

- A* (A-star) is an informed search algorithm used to find the **shortest (least cost) path** from a start node to a goal node.
- It combines the advantages of Uniform Cost Search (uses path cost) and Greedy Best First Search (uses heuristic).
- It uses an evaluation function $f(n) = g(n) + h(n)$ to decide which node to expand next.
- A* is **complete and optimal** if the heuristic is **admissible** (never overestimates the true cost).

2 Evaluation Function

$$f(n) = g(n) + h(n)$$

- $g(n)$: Actual cost from start node to node n .
- $h(n)$: Heuristic cost (estimated cost from n to goal).
- $f(n)$: Total estimated cost of the best path through n .

Node with the lowest $f(n)$ is expanded first (ties are usually broken using lower $h(n)$ or lower $g(n)$).

3 Algorithm Steps

- Place the start node in the open list (priority queue) with $f(\text{start}) = g(\text{start}) + h(\text{start})$.
- Repeat until goal node is found or open list is empty:
 - Select the node n with the lowest $f(n)$ from open list.
 - If n is the goal node, return the path.
 - Move n to the closed list.
 - For each unvisited neighbor m of n :
 - Calculate $g(m)$, $h(m)$ and $f(m)$.
 - If m is not in open list or a better path is found, add/update m in open list (with parent n).
- If open list becomes empty, no path exists.

4 Properties

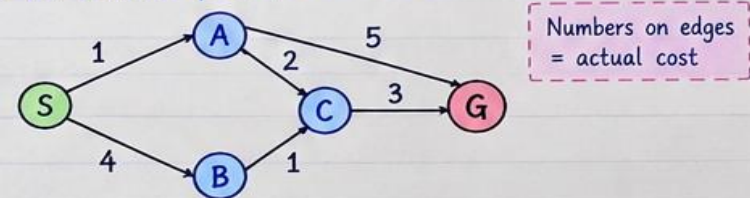
- Complete**: Yes (if branching factor is finite)
- Optimal**: Yes (if heuristic is admissible)
- Time Complexity**: Exponential (worst case)
- Space Complexity**: Exponential
- Uses**: Path finding, route planning, game AI, robotics, etc.

5 Heuristic Function

- $h(n)$ estimates the cost from node n to goal.
- Common heuristics (for grid problems):
 - Manhattan distance: $h(n) = |x_1 - x_2| + |y_1 - y_2|$
 - Euclidean distance: $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
- For optimality, heuristic should be **admissible** (never overestimates the true cost).

6 Example 1: Graph Search

Find the shortest path from S to G.



Numbers on edges
= actual cost

Heuristic values to goal ($h(n)$):

Node	S	A	B	C	G
$h(n)$	6	4	4	2	0

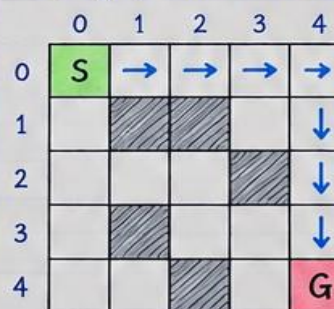
Step-by-step execution:

Step	Current Node (expanded)	Open List (node : f)	Closed List
1	S	A:5, B:8	S
2	A	C:5, B:8, G:6	S, A
3	C	G:5, B:8	S, A, C
4	G (goal)	-	S, A, C, G

Result: Optimal path = $S \rightarrow A \rightarrow C \rightarrow G$
Total cost = $1 + 2 + 3 = 6$

7 Example 2: Grid Path Finding (Manhattan Distance)

Find the shortest path from start (S) to goal (G) in a 5x5 grid (with obstacles). Movement cost = 1 in four directions (up, down, left, right). Heuristic: Manhattan distance.



Start: (0,0)

Goal: (4,4)

Heuristic: $h(n) = |x_1 - x_2| + |y_1 - y_2|$

Example: For node (2,3),

$$h(2,3) = |2-4| + |3-4| = 2 + 1 = 3$$

One optimal path found by A*:

$(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (0,3) \rightarrow (0,4) \rightarrow (1,4) \rightarrow (2,4) \rightarrow (3,4) \rightarrow (4,4)$

Total cost = 8

8 Example 3: Comparing Search Algorithms

Same grid as above. Number of nodes expanded:

Algorithm	Nodes Expanded (approx.)
BFS	20
Uniform Cost Search	18
Greedy Best First Search	10 (but not always optimal)
A* (Manhattan heuristic)	9 (optimal path)

"A* expands fewer nodes than BFS/UCS and still finds the optimal path."

9 Applications

- GPS and map navigation (Google Maps)
- Video game path finding (e.g., NPC movement)
- Robotics and autonomous systems
- Network routing, puzzle solving, etc.

**Smarter Search
Better Paths
Brighter Solutions!**